ml_regularization_ridge_lasso

In [18]:
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
%matplotlib inline

from sklearn import preprocessing
from sklearn.preprocessing import PolynomialFeatures
from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import r2_score
```

In [3]:
```
data = pd.read_csv(r"C:\Users\Avinash\Downloads\car-mpg.csv")
data.head()
```

Out[3]:

	mpg	cyl	disp	hp	wt	acc	yr	origin	car_type	car_name
0	18.0	8	307.0	130	3504	12.0	70	1	0	chevrolet chevelle malibu
1	15.0	8	350.0	165	3693	11.5	70	1	0	buick skylark 320
2	18.0	8	318.0	150	3436	11.0	70	1	0	plymouth satellite
3	16.0	8	304.0	150	3433	12.0	70	1	0	amc rebel sst
4	17.0	8	302.0	140	3449	10.5	70	1	0	ford torino

In [4]:
```
data = data.drop(["car_name"],axis=1)                                    # a
data["origin"] = data["origin"].replace({1:"america",2:"europe",3:"asia"})   # r
data = pd.get_dummies(data,columns=["origin"],dtype=int)                 # e
data = data.replace("?",np.nan)                                          # r
                                                                         # i
```

In [6]:
```
# Convert all columns to numeric where possible
data = data.apply(pd.to_numeric, errors='ignore')

# Fill missing values with median only for numeric columns
numeric_cols = data.select_dtypes(include=[np.number]).columns
data[numeric_cols] = data[numeric_cols].apply(lambda x: x.fillna(x.median()))
```

```
C:\Users\Avinash\AppData\Local\Temp\ipykernel_15416\4166339649.py:2: FutureWarnin
g: errors='ignore' is deprecated and will raise in a future version. Use to_numer
ic without passing `errors` and catch exceptions explicitly instead
  data = data.apply(pd.to_numeric, errors='ignore')
```

In [9]:
```
data.head()
```

Out[9]:

	mpg	cyl	disp	hp	wt	acc	yr	car_type	origin_america	origin_asia	origin_
0	18.0	8	307.0	130.0	3504	12.0	70	0	1	0	
1	15.0	8	350.0	165.0	3693	11.5	70	0	1	0	
2	18.0	8	318.0	150.0	3436	11.0	70	0	1	0	
3	16.0	8	304.0	150.0	3433	12.0	70	0	1	0	
4	17.0	8	302.0	140.0	3449	10.5	70	0	1	0	

Model Building

In [11]:
```
# Once scaled data becomes numpy array then it becomes difficult to devide. Henc
# seperate dependent and independent variable

x = data.drop(["mpg"],axis=1)
y= data[["mpg"]]
```

In [12]:
```
# feature scaling
# Scaling the data

x_s = preprocessing.scale(x)
x_s = pd.DataFrame(x_s, columns = x.columns) # converting scaled data into dataf

y_s = preprocessing.scale(y)
y_s = pd.DataFrame(y_s,columns = y.columns)   # ideally train, test data should
```

In [13]:
```
# Split into train and test data
x_train, x_test, y_train, y_test = train_test_split(x_s, y_s , test_size=0.3, ra
x_train.shape
```

Out[13]: (278, 10)

Simple Linear Model

In [14]:
```
# Fit simple linear model and find coefficient

regression_model = LinearRegression()
regression_model.fit(x_train, y_train)

for idx, col_name in enumerate(x_train.columns):
    print("The coeficient for {} is {}".format(col_name,regression_model.coef_[0

intercept = regression_model.intercept_
print("The intercept is {}".format(intercept))


# As it's a multiple linear regression model have multiple coeffiecient. regress
# If y train is 2D this array is also 2D to access each coefiecient from it used
```

```
The coeficient for cyl is 0.247444797589467
The coeficient for disp is 0.2883821544609875
The coeficient for hp is -0.1899034268715285
The coeficient for wt is -0.6732229065111777
The coeficient for acc is 0.06754501540688178
The coeficient for yr is 0.3446364072117276
The coeficient for car_type is 0.3149149154003767
The coeficient for origin_america is -0.07682943694882893
The coeficient for origin_asia is 0.0633604889661999
The coeficient for origin_europe is 0.03128335735147521
The intercept is [-0.01950047]
```

Regularized Ridge Rgression

In [19]:
```
# Alpa factor is a lambda(penalty term) which helps to reduce magnitude of coeff

ridge_model = Ridge(alpha = 0.3)
ridge_model.fit(x_train, y_train)

print("Ridge model coef: {}".format(ridge_model.coef_))

# As data has 10 column , 10 coefficients appear here
```

```
Ridge model coef: [[ 0.24424435  0.27853222 -0.18980689 -0.66458446  0.06588077
0.34396213
   0.31169746 -0.07642734  0.06333336  0.03080065]]
```

Regularized Lasso Rgression

In [20]:
```
# Alpa factor is a lambda(penalty term) which helps to reduce magnitude of coeff

lasso_model = Lasso(alpha = 0.3)
lasso_model.fit(x_train, y_train)

print("Lasso model coef: {}".format(lasso_model.coef_))

# As data has 10 column , 10 coefficients appear here
# many coefficient are reduced to zero  and y = m1x1 + m2x2 + m3x3 + ...+ mnxn
```

```
Lasso model coef: [-0.         -0.         -0.00057227 -0.45987732  0.
0.12553912
  0.02262423 -0.          0.          0.        ]
```

Score Comparison

In [ ]:
```
# Model score = r^2 or coefiecient of determinant
# r^2 = 1 - (SSR/SST) = Regression Error / SST

# Simple linear model
print("R^2 for Simple linear model")
print(regression_model.score(x_train, y_train))
print(regression_model.score(x_test, y_test))

# Ridge
print("R^2 for Ridge model")
```

```
print(ridege_model.score(x_train, y_train))
print(ridge_model.score(x_test, y_test))

# Lasso
print("R^2 for Lasso model")
print(lasso_model.score(x_train, y_train))
print(lasso_model.score(x_test, y_test))
```

If you wish to further compute polynomial features, you can use the below code. # poly = PolynomialFeatures(degree = 2, interaction_only = True) #Fit calculates u and std dev while transform applies the transformation to a particular set of examples #Here fit_transform helps to fit and transform the X_s #Hence type(X_poly) is numpy.array while type(X_s) is pandas.DataFrame #X_poly = poly.fit_transform(X_s) #Similarly capture the coefficients and intercepts of this polynomial feature model

Model Parameter Tunning

In [21]:
```
# R^2 increases with addition of more attribute irrespective of it's usefulness
# Hence we use Adjusted R^2 which removes the statistical chance that improves R
# sickit does not provide facility to work with adjusted R^2 , so we wuse statsm
# This library requires x and y in single dataframe.

data_train_test = pd.concat([x_train, y_train], axis = 1)
data_train_test.head()
```

Out[21]:

	cyl	disp	hp	wt	acc	yr	car_type	origin_
230	1.498191	1.503514	1.720935	1.412400	-1.513346	0.268063	-1.062235	0
357	-0.856321	-0.714680	-0.112746	-0.420234	-0.278877	1.351199	0.941412	-1
140	1.498191	1.061796	1.197027	1.521175	-0.024722	-0.544290	-1.062235	0
22	-0.856321	-0.858718	-0.243723	-0.703997	0.701436	-1.627426	0.941412	-1
250	1.498191	1.196232	0.935072	0.903991	-0.859804	0.538847	-1.062235	0

In [22]:
```
import statsmodels.formula.api as smf
ols1 = smf.ols(formula = "mpg ~ cyl+disp+hp+acc+yr+car_type+origin_america+origi
ols1.params
```

Out[22]:
```
Intercept        -0.022857
cyl               0.276885
disp             -0.148958
hp               -0.489863
acc              -0.084483
yr                0.306270
car_type          0.390824
origin_america   -0.071211
origin_europe     0.004731
origin_asia       0.081888
dtype: float64
```

In [24]:
```
print(ols1.summary())
```

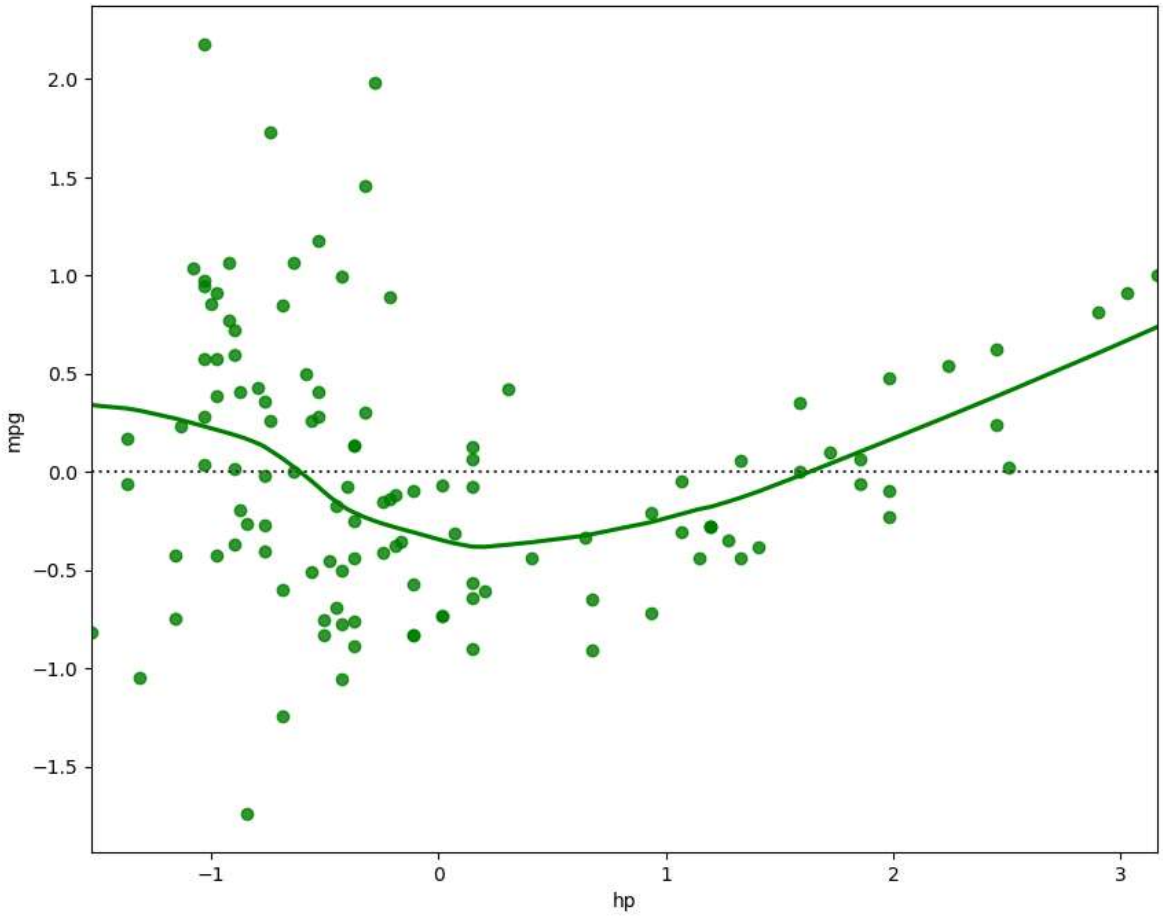
```
                            OLS Regression Results
==============================================================================
Dep. Variable:                    mpg   R-squared:                       0.793
Model:                            OLS   Adj. R-squared:                  0.787
Method:                 Least Squares   F-statistic:                     128.6
Date:                Mon, 23 Jun 2025   Prob (F-statistic):           2.49e-87
Time:                        16:21:00   Log-Likelihood:                -172.58
No. Observations:                 278   AIC:                             363.2
Df Residuals:                     269   BIC:                             395.8
Df Model:                           8
Covariance Type:            nonrobust
==============================================================================
=
                   coef    std err          t      P>|t|      [0.025      0.97
5]
------------------------------------------------------------------------------
-
Intercept         -0.0229      0.028     -0.831      0.407      -0.077       0.03
1
cyl                0.2769      0.115      2.412      0.017       0.051       0.50
3
disp              -0.1490      0.120     -1.246      0.214      -0.384       0.08
6
hp                -0.4899      0.077     -6.341      0.000      -0.642      -0.33
8
acc               -0.0845      0.040     -2.115      0.035      -0.163      -0.00
6
yr                 0.3063      0.031      9.888      0.000       0.245       0.36
7
car_type           0.3908      0.072      5.458      0.000       0.250       0.53
2
origin_america    -0.0712      0.022     -3.187      0.002      -0.115      -0.02
7
origin_europe      0.0047      0.022      0.211      0.833      -0.039       0.04
9
origin_asia        0.0819      0.022      3.646      0.000       0.038       0.12
6
==============================================================================
Omnibus:                       35.905   Durbin-Watson:                   1.814
Prob(Omnibus):                  0.000   Jarque-Bera (JB):               55.086
Skew:                           0.785   Prob(JB):                     1.09e-12
Kurtosis:                       4.514   Cond. No.                     9.74e+15
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly
specified.
[2] The smallest eigenvalue is 1.42e-29. This might indicate that there are
strong multicollinearity problems or that the design matrix is singular.
```

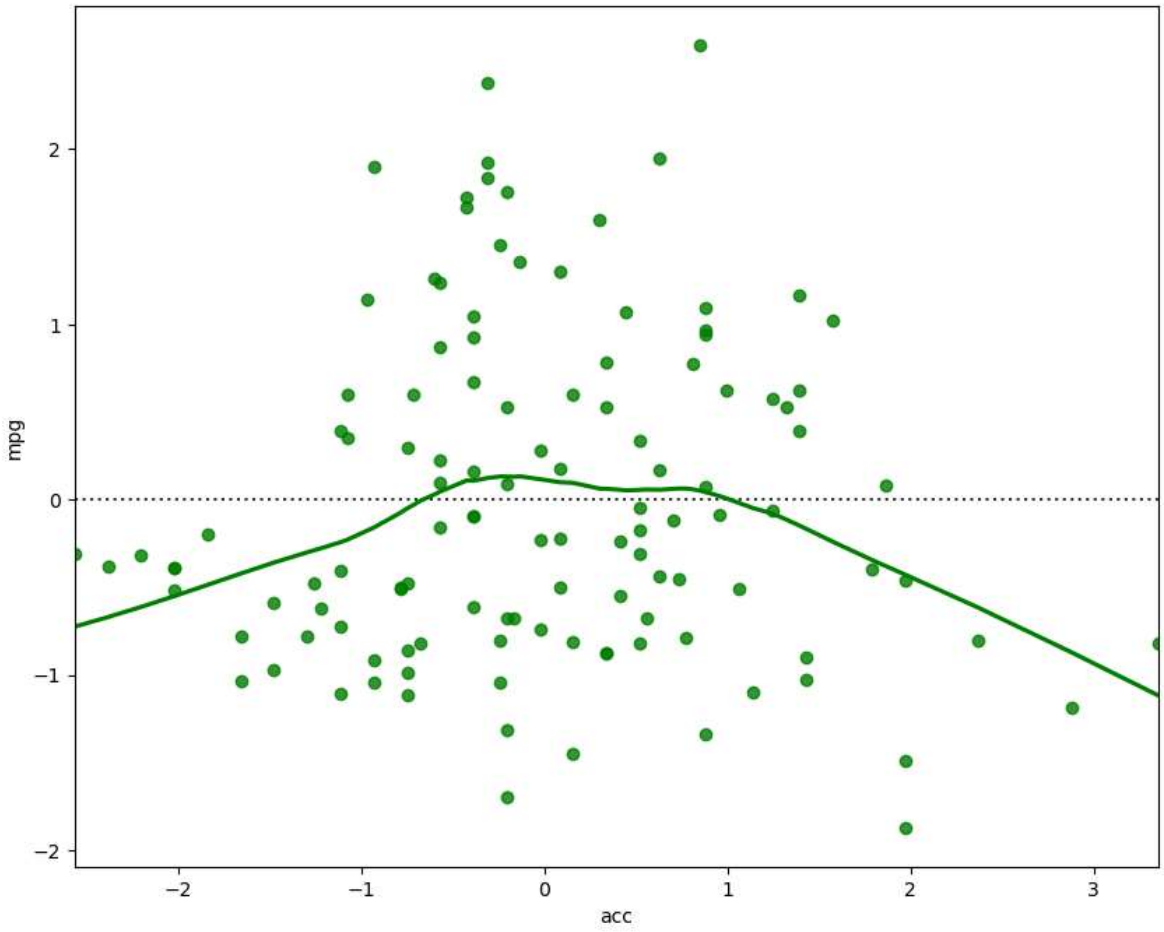
In [25]:

```
 # SSE = (actual y - predicted y)^2 = (y_test - y_pred)^2

mse = np.mean((regression_model.predict(x_test) - y_test)**2)

# root mean_sq_error is standard deviation i.e. avg varience between predicted a
import math
rmse = math.sqrt(mse)
print("Root Mean Squared Error: {}".format(rmse))
```

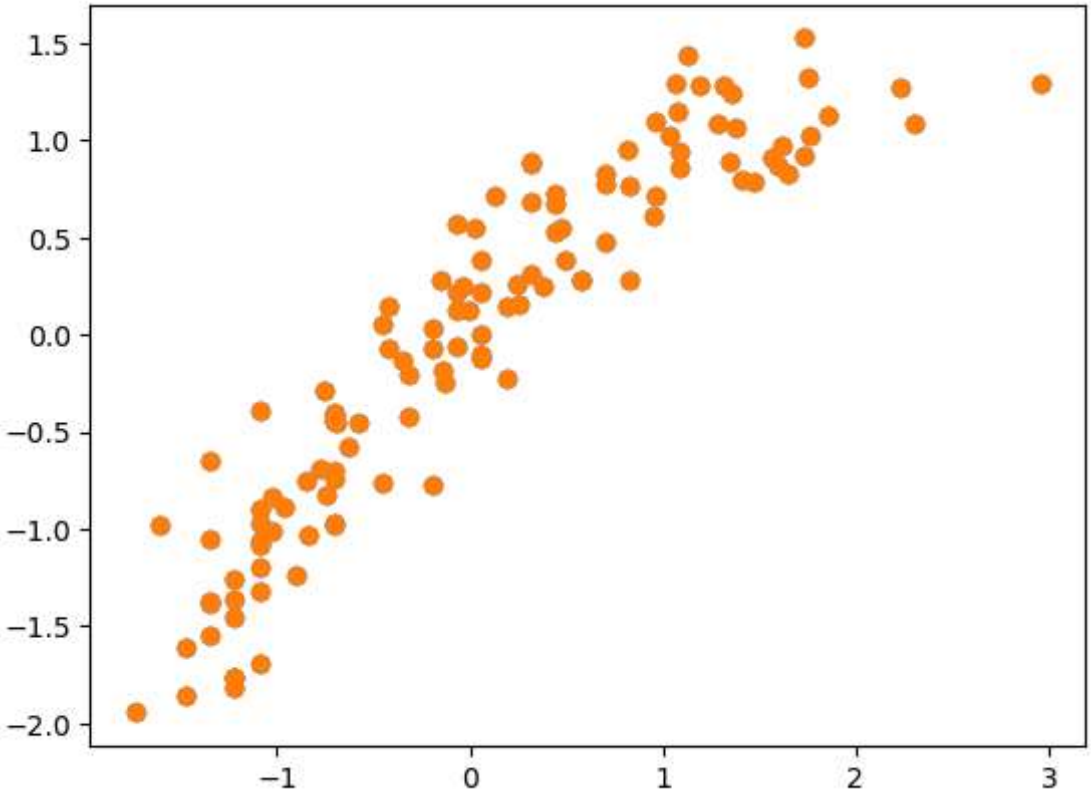
```
Root Mean Squared Error: 0.4045366587848482
```

In [28]:
```
# Is OLS good model

fig = plt.figure(figsize=(10,8))
sns.residplot(x = x_test["hp"],y=y_test["mpg"],color="green", lowess = True)

fig = plt.figure(figsize=(10,8))
sns.residplot(x = x_test["acc"],y=y_test["mpg"],color="green", lowess = True)
plt.show()
```





In [30]:
```
# predict mileage (mpg) for a set of attributes not in the training or test set
y_pred = regression_model.predict(x_test)

plt.scatter(y_test["mpg"],y_pred)
plt.show()
# for a good model predicted value should be close to actual values leading to h
```



Inference Both ridge and lasso regularization perform very well on this data. Ridge gives better score. The above scatter plot depicts the correlation between actual and predicted mpg values